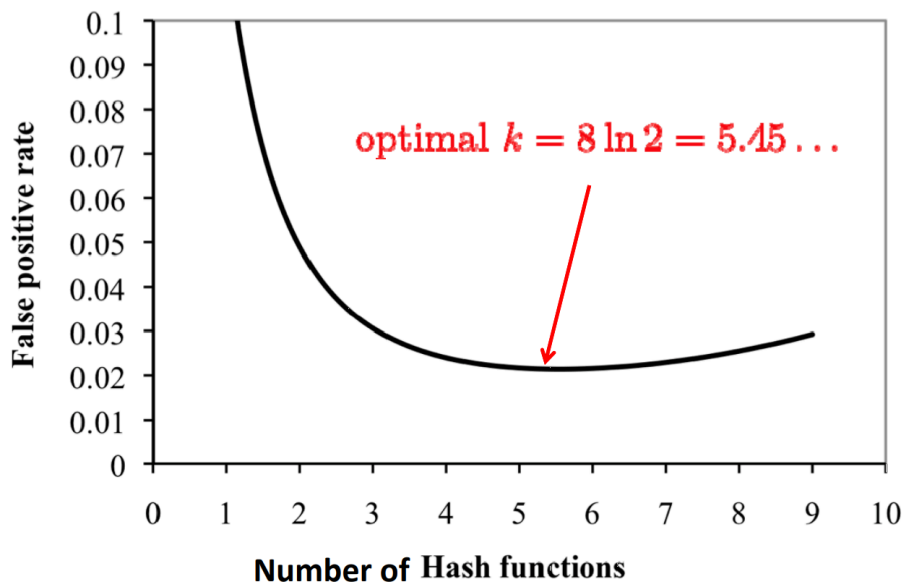
False positive probability (credit: Simon S. Lam)

- The size of k is a trade-off:
  - A higher k increase the number of hash functions that might map to 0
  - But also ”depletes” the available 0 slots in the bloom filter
- Optimal k: $k = \frac{m}{n} ln2$

Number of bits per member $\frac{m}{n} = 8$



Bloom filter calculator (https://hur.st/bloomfilter/)

Bloom Filter Calculator

Bloom filters are space-efficient probablistic data structures used to test whether an element is a member of a set.

They're surprisingly simple: take an array of **m** bits, and for up to **n** different elements, either test or set **k** bits using positions chosen using hash functions. If all bits are set, the element *probably* already exists, with a false positive rate of **p**; if any of the bits are not set, the element *certainly* does not exist.

Bloom filters find a wide range of uses, including tracking which articles you've read, speeding up Bitcoin clients, detecting malicious web sites, and improving the performance of caches.

This page will help you choose an optimal size for your filter, or explore how the different parameters interact.

n

Number of items in the filter (optionally with SI units: k, M, G, T, P, E, Z, Y)

4000

p

Probability of false positives, fraction between 0 and 1 or a number indicating 1-in-p

1.0E-7

m

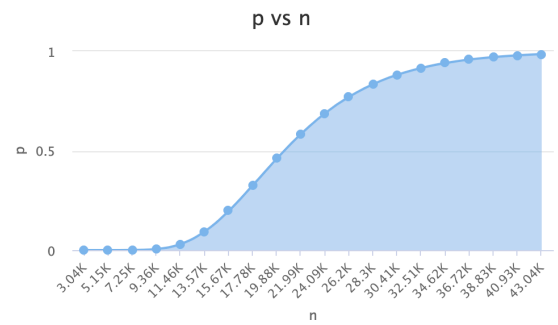
Number of bits in the filter (or a size with KB, KiB, MB, Mb, GiB, etc)

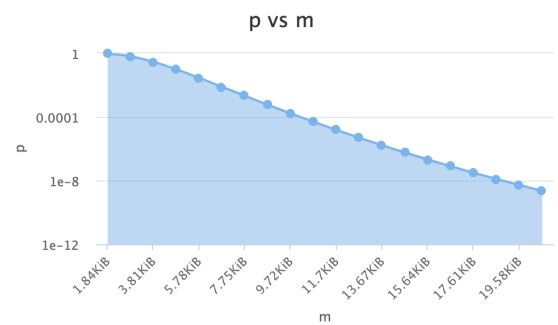
k

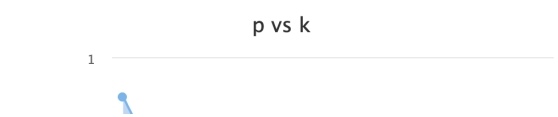
Number of hash functions

Submit

n = 4,000
p = 0.0000001 (1 in 9,994,297)
m = 134,191 (16.38KiB)
k = 23





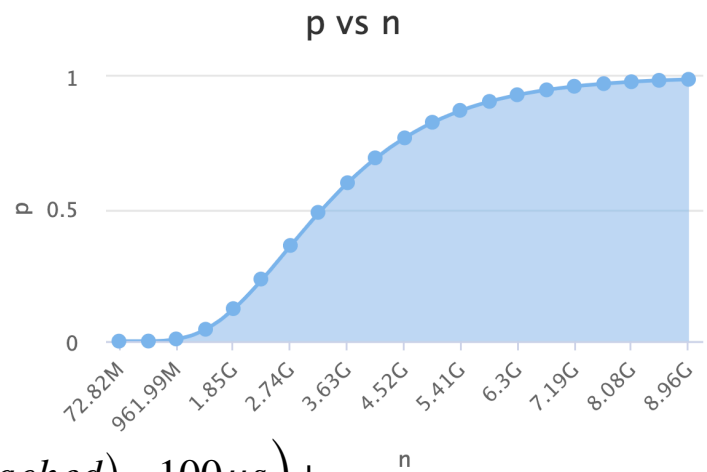


Trade off memory space vs. speed

- Larger bloom filter:
  - Reduces false positives → fewer reads to disk → lower latency, higher throughput
- But…
  - Takes up more space in memory → less space for caching database index entries in memory → higher chance of going to disk → higher latency, lower throughput

Example of space vs. speed trade off

- Example: database has 2GB of memory for caching and 100GB of flash
  - Needs to support 1 billion entries (each ~100B)
- Flash access is 100us, memory access is 100ns
- Which bloom filter parameters would you choose?
  1. 600MB bloom filter:
     - Bloom filter false positive rate of 9%
     - 1.4GB of DRAM left → index cache hit rate of 90%
  2. 1.1GB bloom filter:
     - Bloom filter false positive rate of 1%
     - 0.9GB of DRAM left → index cache hit rate of 70%



- 60% of requests return object does not exist:
- $$\boldsymbol{avg\ latency} = \mathbb{P}(exists) \bullet \left(\mathbb{P}\left(cached\right) \bullet 100ns + \mathbb{P}\left(not_cached\right) \bullet 100\mu s\right) +$$

$$\mathbb{P}(doesn't_exist) \bullet \left(\mathbb{P}\left(true\ negative\right) \bullet 100ns + \mathbb{P}\left(false\ positive\right) \bullet 100\mu s\right)$$

- Scenario 1
  - 0.4 * (0.9 * 100ns + 0.1 * 100us) + 0.6 * (0.91 * 100ns + 0.09 * 100us) = **9.49us**
- Scenario 2:
  - **12.64us**

Other issues with bloom filters

- Can get “depleted” over time
- Need to estimate number of unique entries in advance
- Do not support deletes!
  - Why?

- Improvements: counting bloom filters, cuckoo filters, learned bloom filters, elastic bloom filters… This is a hot research area!

Algorithmic Nuggets in Content Delivery

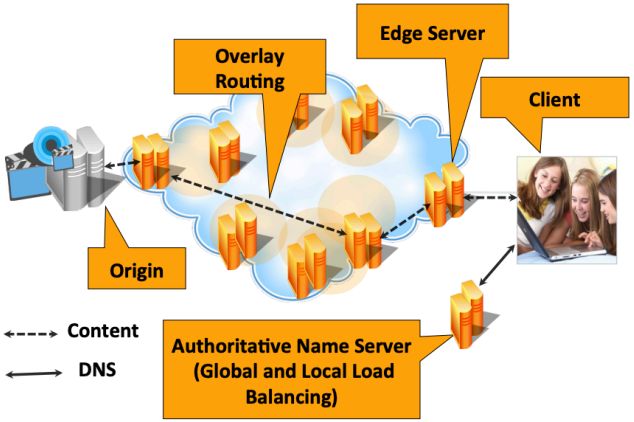
Bruce M. Maggs
Duke and Akamai
bmm@cs.duke.edu

Ramesh K. Sitaraman
UMass, Amherst and Akamai
ramesh@cs.umass.edu

This article is an editorial note submitted to CCR. It has NOT been peer reviewed.
The authors take full responsibility for this article's technical content. Comments can be posted through CCR Online.

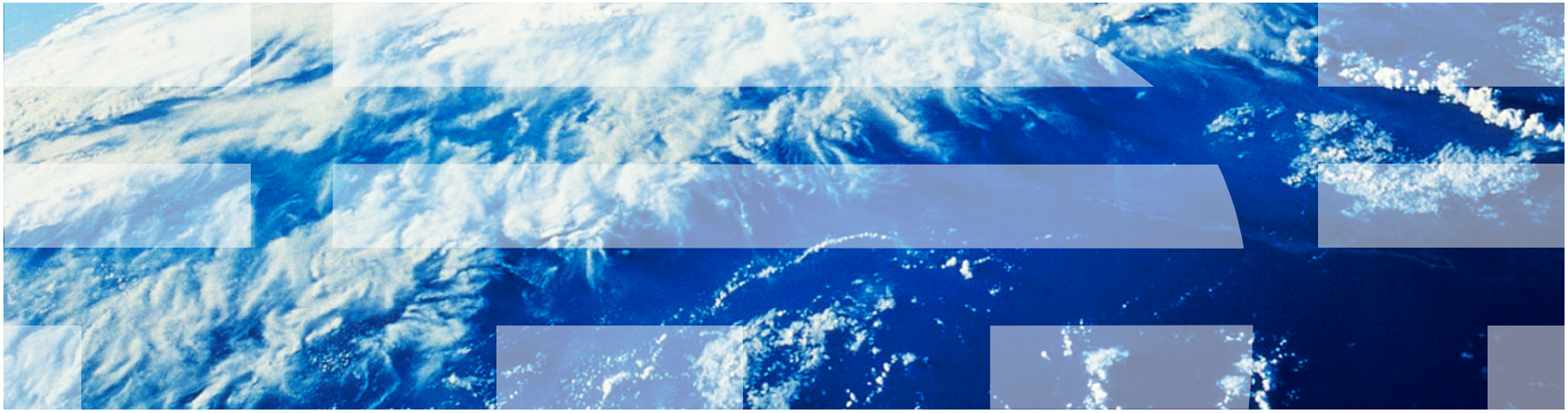
ABSTRACT

This paper "peeks under the covers" at the subsystems that provide the basic functionality of a leading content delivery network. Based on our experiences in building one of the largest distributed systems in the world, we illustrate how sophisticated algorithmic research has been adapted to balance the load between and within server clusters, manage the caches on servers, select paths through an overlay routing network, and elect leaders in various contexts. In each instance, we first explain the theory underlying the algorithms, then introduce practical considerations not captured by the theoretical models, and finally describe what is implemented in practice. Through these examples, we highlight the role of algorithmic research in the design of complex networked systems. The paper also illustrates the close synergy that exists between research and industry where research ideas cross over into products and product requirements drive future research.



https://people.cs.umass.edu/~ramesh/Site/PUBLICATIONS_files/CCRpaper_1.pdf

Caching



Motivation

- Many systems use caches for reducing work/transport duplication
  - Databases cache frequently accessed keys in memory
  - Computers use memory as a cache
  - Content delivery networks are a cache
  - The file system has a cache, called a page cache
  - …

- Big data systems even use dedicated cache servers (key-value caches)
  - All data is stored in memory (does not use disk)
  - Before going to database, check key-value cache
  - Examples:
    - Memcached
    - Redis
    - Amazon Elasticache

Important questions for caching

- Assignment: where do you put the data?
  - In memory? How? Optimize read speed
- Consistency: what happens when data is updated/deleted?
  - Passive—cache entry expires
  - Active—cache entry is invalidated
- Eviction: who do you kick out?
  - Cache space is expensive—otherwise, we would put everything there
  - What do you do when you run out of space?

Standard Eviction Policy Model

- Cache has fixed size (C)
  - Each item has the same size (1)
  - Each item has a unique key
- Cache is full
- Requests come in
  - If the request is for an item that exists in the cache, it is read
  - If the request is for an item that does not exist in the cache, it is fetched (from a second storage system, e.g., a database), and another item is evicted to make room for it
- Eviction policy decides which object to evict when a new item is inserted

Metric for success

- Overall goal might be total latency or throughput (usually latency)
  - E.g., what's the performance of the total system given a certain eviction policy
- Typically average latency is equivalent to cache hit rate
  - Over time window, percentage of read requests whose objects existed in the cache
- Example (includes the network latency):
  - Average latency to flash database: 5 millisecond
  - Average latency to in-memory key-value cache: 100 microseconds
  - Average latency (in microseconds) when cache hit rate is 98%:
    - 0.98 * 100 + 0.02 * 5000 = 198 microseconds
  - Average latency (in microseconds) when cache hit rate is 99%:
    - 0.99 * 100 + 0.01 * 5000 = 149 microseconds
  - 1% cache hit rate increase improves latency by 25%!

Common eviction policies

- FIFO (first in first out)
- LRU (least recently used)
- CLOCK (approximates LRU)
- LFU (least frequently used)
- OPT (Belady's algorithm: knows the future)